



CredShields

Smart Contract Audit

May 14, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Bitcoin Limited Edition on May 14, 2026. A retest was performed on May 14, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Bitcoin Limited Edition



Table of Contents

Table of Contents	2
1. Executive Summary -----	3
State of Security	4
2. The Methodology -----	5
2.1 Preparation Phase	5
2.1.1 Scope	5
2.1.2 Documentation	5
2.1.3 Audit Goals	6
2.2 Retesting Phase	6
2.3 Vulnerability classification and severity	6
2.4 CredShields staff	8
3. Findings Summary -----	9
3.1 Findings Overview	9
3.1.1 Vulnerability Summary	9
5. Bug Reports -----	10
Bug ID #L001 [Acknowledged]	10
Floating and Outdated Pragma	10
Bug ID #L002 [Acknowledged]	12
Use of transfer() for ETH Forwarding	12
Bug ID #I001 [Fixed]	13
Use Ownable2Step	13
Bug ID #I002 [Acknowledged]	14
Missing zero address validations	14
Bug ID #I003 [Acknowledged]	15
Trailing Space in Token Name	15
6. The Disclosure -----	16



1. Executive Summary -----

Bitcoin Limited Edition engaged CredShields to perform a smart contract audit on May 14, 2026. During this timeframe, 5 vulnerabilities were identified. **A retest was performed on May 14, 2026, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Bitcoin Limited Edition" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
BTCLE Token Contract	0	0	0	2	3	0	5

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in BTCLE Token Contract's scope during the testing window while abiding by the policies set forth by Bitcoin Limited Edition's team.



State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Bitcoin Limited Edition's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Bitcoin Limited Edition can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Bitcoin Limited Edition can future-proof its security posture and protect its assets.



2. The Methodology -----

Bitcoin Limited Edition engaged CredShields to perform a BTCLE Token Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window on May 14, 2026 was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Audited Scope: https://bscscan.com/token/0x9d2144328e1d618f54cd38540f5ee50671f6a208

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.



2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the [OWASP Smart Contract Security](#) projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Bitcoin Limited Edition is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat



agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	● Medium	● High	● Critical
	MEDIUM	● Low	● Medium	● High
	LOW	● None	● Low	● Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. Informational

We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. Low

Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. Medium

Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities



can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. High

High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. Critical

Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. Gas

To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- **Shashank, Co-founder CredShields** shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.



3. Findings Summary -----

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 5 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Floating and Outdated Pragma	Low	Floating Pragma (SCWE-060)	Acknowledged
Use of transfer() for ETH Forwarding	Low	Best Practice	Acknowledged
Use Ownable2Step	Informational	Missing Best Practices	Fixed
Missing zero address validations	Informational	Missing Input Validation (SC04-Lack Of Input Validation)	Acknowledged
Trailing Space in Token Name	Informational	Misconfiguration	Acknowledged

Table: Findings and Remediations



5. Bug Reports -----

Bug ID #L001[Acknowledged]

Floating and Outdated Pragma

Vulnerability Type

Floating Pragma ([SCWE-060](#))

Severity

Low

Description

Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.

The contract allowed floating or unlocked pragma to be used, i.e., $\geq 0.8.0$. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- [0x9d2144328e1d618f54cd38540f5ee50671f6a208#code#F1#L3](#)

Impacts

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.

Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.

The likelihood of exploitation is low.

Remediation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.34 pragma version

Reference: <https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/>

Retest



Client's Comment: Contract is already deployed and ownership renounced, the compiler version used during the initial deployment is now fixed for the life of the contract. No action possible due to contract renunciation.



Bug ID #L002 [Acknowledged]

Use of transfer() for ETH Forwarding

Vulnerability Type

Best Practice

Severity

Low

Description

The contract uses transfer() to forward ETH, which imposes a hardcoded 2300 gas stipend on the recipient. Since EIP-1884 increased the cost of certain opcodes, this limit is no longer considered safe and may cause unexpected reverts when the recipient is a smart contract.

Affected Code

- [0x9d2144328e1d618f54cd38540f5ee50671f6a208#code#F1#L17](#)

Impacts

ETH transfer may fail if the recipient is a smart contract, causing the transaction to revert.

Remediation

Use low-level `call()` instead, which forwards all available gas and works safely for both EOAs and contracts

Retest

Client's Comment: Cannot change to `.call()` as logic is locked on-chain. Future integrators advised to ensure receiving contracts handle the current gas limit.



Bug ID #I001[Fixed]

Use Ownable2Step

Vulnerability Type

Missing Best Practices

Severity

Informational

Description

The "Ownable2Step" pattern is an improvement over the traditional "Ownable" pattern, designed to enhance the security of ownership transfer functionality in a smart contract. Unlike the original "Ownable" pattern, where ownership can be transferred directly to a specified address, the "Ownable2Step" pattern introduces an additional step in the ownership transfer process. Ownership transfer only completes when the proposed new owner explicitly accepts the ownership, mitigating the risk of accidental or unintended ownership transfers to mistyped addresses.

Affected Code

- [0x9d2144328e1d618f54cd38540f5ee50671f6a208#code#F1#L5](#)

Impacts

Without the "Ownable2Step" pattern, the contract owner might inadvertently transfer ownership to an unintended or mistyped address, potentially leading to a loss of control over the contract. By adopting the "Ownable2Step" pattern, the smart contract becomes more resilient against external attacks aimed at seizing ownership or manipulating the contract's behavior.

Remediation

It is recommended to use either Ownable2Step or Ownable2StepUpgradeable depending on the smart contract.

Retest:

Client's Comment: Ownership has already been renounced. No further transfers of ownership can occur.



Bug ID #I002 [Acknowledged]

Missing zero address validations

Vulnerability Type

Missing Input Validation ([SC04-Lack Of Input Validation](#))

Severity

Informational

Description:

The contracts were found to be setting new addresses without proper validations for zero addresses.

Address type parameters should include a zero-address check otherwise contract functionality may become inaccessible or tokens burned forever.

Depending on the logic of the contract, this could prove fatal and the users or the contracts could lose their funds, or the ownership of the contract could be lost forever.

Affected Code

- [0x9d2144328e1d618f54cd38540f5ee50671f6a208#code#F1#L14](#)

Impacts

If address type parameters do not include a zero-address check, contract functionality may become unavailable or tokens may be burned permanently.

Remediation

Add a zero address validation to all the functions where addresses are being set.

Retest

Client's Comment: Code cannot be updated. Strict front-end validations and internal operational procedures are in place to prevent zero-address inputs.



Bug ID #I003 [Acknowledged]

Trailing Space in Token Name

Vulnerability Type

Informational

Severity

Informational

Description

The token name is defined as "Bitcoin Limited Edition " with a trailing space. This value is stored on-chain and returned by name().

Affected Code

- [0x9d2144328e1d618f54cd38540f5ee50671f6a208#code#F1#L15](#)

Impacts

DEX UIs, block explorers, and off-chain integrations that exact-match or trim token names may display incorrectly or fail to recognize the token.

Remediation

Remove the trailing space:

- `BEP20Base("Bitcoin Limited Edition ", "BTCLE", 18, 0x312f313730393733392f4f);`
- `BEP20Base("Bitcoin Limited Edition", "BTCLE", 18, 0x312f313730393733392f4f);`

Retest

Client's Comment: Trailing space is a permanent on-chain characteristic. All marketing materials and documentation will be aligned to match the on-chain string.



6. The Disclosure -----

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.



Your **Secure Future** Starts Here



At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets



 SCS Advocates

